

Here's a polished, detailed prompt you can directly paste into Claude Code, Claude Sonnet, Gemini, Cursor, or any coding LLM.

ROLE

You are a **Senior C++ Software Engineer, Senior UI/UX Designer**, and **Object-Oriented Programming Instructor**. Your task is to build a **complete academic-level desktop application** that can score full marks in an Object-Oriented Programming (ENCT151) coursework submission.

The application must follow professional software engineering practices, clean architecture, SOLID principles where appropriate, modern C++ standards (C++17 or newer), and extensive object-oriented design.

This is **NOT** a simple CRUD project.

It should feel like a professional restaurant POS and Restaurant Management System.

PROJECT NAME

AluChop

Restaurant Management System

OBJECTIVE

Create a fully functional **Restaurant Management System** named **AluChop** using **C++** with a **modern GUI**.

The application should resemble lightweight restaurant management software used in cafés and restaurants.

The GUI should be minimalistic, premium, modern, and use a **Sage Green aesthetic**.

The design inspiration should resemble:

- Modern POS software
- Restaurant dashboards
- Apple Human Interface

- Clean Material Design
 - Glassmorphism
 - Soft rounded cards
 - Elegant typography
 - Plenty of whitespace
-

VERY IMPORTANT

I have attached:

- Restaurant Menu Images
- Restaurant Software Inspiration
- OOP Coursework PDF

Analyze ALL attached images and the attached PDF before writing a single line of code.

Especially analyze the PDF carefully.

Every topic inside the coursework **must appear somewhere inside the project**, not artificially, but naturally.

The goal is to demonstrate mastery of the complete syllabus while still producing a real software application. The coursework expects students to submit a programming project demonstrating the application of object-oriented programming concepts.

2025_10_09_08_55_17

GUI REQUIREMENTS

Use any good modern C++ GUI library such as:

Preferred:

- Qt 6

OR

- Dear ImGui

OR

- wxWidgets

OR

- FLTK

(Choose whichever results in the cleanest and most professional desktop application.)

DESIGN

Theme:

Sage Green

Suggested Palette

Primary:

#5D7A66

Secondary:

#7E9B84

Accent:

#A8C3A1

Background:

#F6F8F4

Cards:

#FFFFFF

Borders:

#D7E4D2

Success:

#5E9E66

Danger:

#D16464

Text:

#1F2D1F

The GUI should include:

- Rounded corners
- Soft shadows
- Glassmorphism where appropriate

- Smooth animations
- Sidebar navigation
- Minimal icons
- Dashboard widgets
- Elegant tables
- Search bars
- Statistics cards
- Clean typography
- Responsive layouts
- Professional spacing

Avoid clutter.

The interface should feel like modern restaurant management software.

MENU

Use the attached restaurant menu as the official menu database.

Use:

- Item names
- Categories
- Prices

The prices are already inclusive of tax.

Therefore:

DO NOT calculate extra tax.

Store prices exactly in **NPR**.

Categories include:

- Sushi
- Pizza
- Pasta
- Main Course
- Dimsum
- From the Tandoor
- From the Wok
- Bread & Rice
- Dessert

- Drinks
- Beer
- Wine
- Mocktails
- Shots

Preserve pricing in Nepalese Rupees (NPR).

FEATURES

Authentication

Admin Login

Employee Login

Forgot Password

Remember Login

Role-based authentication

Dashboard

Daily Sales

Weekly Sales

Monthly Sales

Popular Items

Inventory Alerts

Reservations

Pending Orders

Revenue Graphs

Customer Count

Restaurant Menu

Browse

Search

Sort

Filter

Categories

Item Images

Descriptions

Pricing

Availability

Order Management

Create Order

Edit Order

Delete Order

Dine In

Takeaway

Delivery

Split Bills

Merge Bills

Order Queue

Kitchen Status

Generate Bill

Print Receipt

Customer Management

Customer Database

Phone

Email

Loyalty Points

Visit History

Favorite Orders

Employee Management

Staff Information

Salary

Attendance

Position

Shift

Performance

Inventory

Ingredients

Stock

Supplier

Low Stock Alerts

Restocking

Expiry Dates

Inventory Usage

Reservation System

Table Booking

Time

Guests

Special Requests

Availability

Billing

Invoice Generation

Receipt

Discounts

Promo Codes

Service Charges

Tax Included

Cash

Card

Digital Wallet

Change Calculation

Reports

Sales

Inventory

Orders

Revenue

Customers

Employees

Charts

CSV Export

PDF Export

Settings

Restaurant Information

Themes

Database Backup

Restore

Export

DATABASE

Use file storage or SQLite.

Persist:

Customers

Employees

Orders

Reservations

Inventory

Sales

Menu

Settings

OOP REQUIREMENTS

This is the most important part.

The project **must naturally incorporate every major concept from the ENCT151 Object-Oriented Programming syllabus**, including the progression from C++ basics through advanced OOP, file handling, templates, and exception handling. 2025_10_09_08_55_17

Demonstrate and clearly organize:

C++ Basics

- Functions
- Function Overloading
- Inline Functions
- Default Arguments
- Pass by Reference
- Return by Reference
- Arrays
- Strings
- Pointers
- Dynamic Memory Allocation
- Structures
- Enumerations
- Namespaces
- Constants

Objects & Classes

Include:

- Multiple Classes
- Objects
- Constructors
- Destructor
- Copy Constructor
- Parameterized Constructor
- Objects as Members
- Object Arrays
- Object Pointers
- Dynamic Objects
- Static Members
- Constant Objects
- Constant Member Functions
- Friend Functions

- Friend Classes
-

Operator Overloading

Include examples such as:

-
-

==

<

<<

|

Assignment

Increment

Any practical operators

Inheritance

Demonstrate:

Single

Multiple

Hierarchical

Multilevel

Hybrid

Public

Protected

Private

Virtual Base Class

Method Overriding

Polymorphism

Virtual Functions

Pure Virtual Functions

Abstract Classes

Runtime Polymorphism

Compile Time Polymorphism

File Handling

Read

Write

Append

Binary Files

ASCII Files

Random Access

Sequential Access

Error Checking

Templates

Function Templates

Class Templates

STL

Vectors

Maps

Queues

Algorithms

Iterators

Exception Handling

Try

Catch

Throw

Multiple Catch

Rethrow

Custom Exceptions

CLASS STRUCTURE

Suggested classes:

Restaurant

Menu

MenuItem

Inventory

Ingredient

Supplier

Customer

Employee

Manager

Chef

Waiter

Admin

Order

OrderItem

Bill

Invoice

Reservation

Payment

Table

Analytics

Authentication

Database

FileManager

Logger

Settings

ThemeManager

ReportGenerator

Notification

Dashboard

SOFTWARE ENGINEERING

Use:

Header Files

Source Files

Namespaces

MVC or layered architecture

Separate GUI

Separate Logic

Separate Database

Separate Models

Meaningful comments

Doxygen-style comments

Consistent naming

Proper encapsulation

Modular code

EXTRA FEATURES

Dark Mode

Light Mode

Search Everywhere

Keyboard Shortcuts

Undo

Redo

Notifications

Auto Save

Settings

Recent Orders

Sales Charts

Animated Dashboard

Loading Screen

Splash Screen

Professional Icons

Sound Effects (optional)

PROJECT STRUCTURE

```
AluChop/  
  
src/  
  
include/  
  
gui/  
  
database/  
  
assets/  
  
fonts/  
  
images/  
  
icons/  
  
menu/  
  
reports/  
  
exports/  
  
docs/  
  
README.md  
  
LICENSE  
  
CMakeLists.txt
```

README

Generate a professional README including:

Project Overview

Features

Installation

Dependencies

Screenshots placeholder

Folder Structure

Class Diagram

UML

OOP Concepts Used

Future Improvements

Credits

DOCUMENTATION

Generate:

UML Class Diagram

Flowchart

Use Case Diagram

ER Diagram (if SQLite used)

Class Relationship Explanation

CODE QUALITY

Follow:

Modern C++17+

No memory leaks

RAII

Smart pointers where appropriate

Meaningful variable names

Proper error handling

Well-formatted code

FINAL TOUCH

At the very bottom of the main application window, create a subtle footer with elegant typography:

Designed & Developed by

Shashank Bhattarai

ACE082BCT078

shashankbhattarai006@gmail.com

Include a tasteful copyright/license line such as:

© 2026 AluChop Restaurant Management System. Developed by Shashank Bhattarai (ACE082BCT078). For academic use as an ENCT151 Object-Oriented Programming coursework project. All rights reserved.

FINAL OUTPUT

Generate a **fully working, compile-ready C++ desktop application**, not pseudocode or partial snippets. The project should be complete, modular, and organized into multiple files with a polished GUI, persistent data storage, comprehensive documentation, and a professional structure suitable for showcasing all required OOP concepts while remaining a realistic restaurant management system.